Reviewer Pack — Mobius-Mertens Cancellation on Fourier Levels vs Total Influ…

Entry 9ea129f53eed · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-16 22:38:29 UTC

Mobius-Mertens Cancellation on Fourier Levels vs Total Influence

Entry ID: `9ea129f53eed` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-16 22:38:29 UTC

1. Conjecture

Field A (mathematical branch): Analytic number theory: the classical Mobius function mu(k) and Mertens-type cancellation applied to integer indices k=|S| labeling Fourier levels of a Boolean function **Field B** (complexity object): Total influence (average sensitivity) of Boolean functions on {-1,+1}^n, i.e. I(f) = sum_S |S|*f_hat(S)^2, the canonical Fourier-analytic complexity measure underlying KKL, Friedgut, and junta theorems

Statement:

> For every non-constant Boolean function f: {-1,+1}^n -> {-1,+1}, define the Mobius-Mertens level sum M(f) := sum_{k=1}^{n} mu(k) * W_k(f), where W_k(f) = sum_{|S|=k} f_hat(S)^2 is the level-k Fourier weight and mu is the number-theoretic Mobius function (mu(1)=1, mu(p)=-1 for primes, mu=0 on non-squarefree integers). Conjecture: |M(f)| * sqrt(1 + I(f)) <= 2 for an absolute constant 2, where I(f) = sum_S |S|*f_hat(S)^2 is total influence. A single instance with |M(f)|sqrt(1+I(f)) > 2 falsifies it.

Rationale (proposer's reasoning):

> The Mobius function exerts sqrt-type cancellation on its partial sums (Mertens phenomenon); applied to the discrete distribution {W_k(f)} on integer levels, it should produce cancellation tied to how spread-out the spectral mass is, which is exactly what total influence I(f) (the mean of the W_k-distribution) controls. A function concentrated at one level k can only avoid cancellation if mu(k) != 0, and the cost of high concentration is forced into I(f), suggesting a sqrt-law trade-off. To my knowledge no paper links classical mu-cancellation to Boolean influence bounds, giving a genuinely new analytic-number-theory x boolean-Fourier bridge.

Taxonomy category: `FOURIER_ANALYTIC` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `173074ff4c685e39`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across n∈{4..10} × 30 seeds × all listed families, compute |M(f)|·√(1+I(f)). SUPPORTED iff every single (n,seed,family) instance yields value ≤ 2.0 (strict universal bound, no tolerance). FALSIFIED iff any one instance exceeds 2.0; report max value and first witness.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	UNCERTAIN	0.90	SAFE	HITS
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.97	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Mobius function Fourier weight Boolean function influence` - `Mertens cancellation level-k Fourier weight total influence Boolean` - `number-theoretic Mobius Boolean Fourier spectrum junta KKL`

Top relevant hits considered: - [http://arxiv.org/abs/2202.01071v5] Autocorrelation of the Mobius Function - [http://arxiv.org/abs/2206.12956v3] Result on the Mobius Function over Shifted Primes - [http://arxiv.org/abs/2003.09703v1] Variance function of boolean additive convolution - [http://arxiv.org/abs/math/0504289v3] Mertens' Proof of Mertens' Theorem - [http://arxiv.org/abs/2406.18700v4] Structure of sparse Boolean functions over Abelian groups, and its application to testing - [http://arxiv.org/abs/1804.03587v1] Symmetries on plabic graphs and associated polytopes - [http://arxiv.org/abs/2410.15822v2] Learning junta distributions, quantum junta states, and QAC^0 circuits

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import sys
import random
import math
import itertools
from collections import defaultdict

def mobius(n):
    if n == 1:
        return 1
    factors = set()
    temp = n
    for i in range(2, int(math.sqrt(temp)) + 1):
        if temp % i == 0:
            factors.add(i)
            while temp % i == 0:
                temp //= i
    if temp > 1:
```

```
        factors.add(temp)
    if len(factors) % 2 == 1:
        return -1
    else:
        return 0

def fast_walsh_hadamard(f):
    n = len(f)
    if n == 1:
        return f
    half = n // 2
    even = fast_walsh_hadamard(f[:half] + f[half:])
    odd = fast_walsh_hadamard(f[half:] + f[:half])
    return [e + o for e, o in zip(even, odd)] + [e - o for e, o in zip(even, odd)]

def generate_function(n, family, seed):
    random.seed(seed)
    if family == "balanced_uniform":
        f = [random.choice([-1, 1]) for _ in range(2**n)]
        while sum(f) != 0:
            f = [random.choice([-1, 1]) for _ in range(2**n)]
        return f
    elif family == "dictators":
        index = random.randint(0, n-1)
        f = [-1] * (2**n)
        for i in range(2**n):
            if (i >> index) & 1:
                f[i] = 1
        return f
    elif family == "AND_n":
        return [1 if all((i >> j) & 1 for j in range(n)) else -1 for i in range(2**n)]
    elif family == "OR_n":
        return [1 if any((i >> j) & 1 for j in range(n)) else -1 for i in range(2**n)]
    elif family == "MAJ_n":
        return [1 if sum((i >> j) & 1 for j in range(n)) > n//2 else -1 for i in range(2**n)]
    elif family == "PARITY_n":
        return [1 if bin(i).count('1') % 2 == 0 else -1 for i in range(2**n)]
    elif family == "k_juntas":
        k = random.randint(1, 3)
        indices = random.sample(range(n), k)
        f = [-1] * (2**n)
        for i in range(2**n):
            if sum((i >> j) & 1 for j in indices) > k//2:
                f[i] = 1
        return f
    elif family == "TRIBES":
        tribes = [random.sample(range(n), n//3) for _ in range(3)]
        f = [-1] * (2**n)
        for i in range(2**n):
            counts = [sum((i >> j) & 1 for j in tribe) for tribe in tribes]
            if max(counts) > n//6:
                f[i] = 1
        return f
    elif family == "addressing":
        index = random.randint(0, n-1)
        f = [-1] * (2**n)
```

```
        for i in range(2**n):
            if (i >> index) & 1:
                f[i] = 1
        return f

def compute_metrics(f, n):
    fourier = fast_walsh_hadamard(f)
    fourier_sq = [x**2 for x in fourier]
    W_k = defaultdict(float)
    I = 0.0
    for S in range(2**n):
        k = bin(S).count('1')
        W_k[k] += fourier_sq[S]
        I += k * fourier_sq[S]
    M = 0.0
    for k in range(1, n+1):
        M += mobius(k) * W_k[k]
    return M, I, W_k

def run_trial(seed):
    random.seed(seed)
    n_values = [4, 5, 6, 7, 8, 9, 10]
    families = ["balanced_uniform", "dictators", "AND_n", "OR_n", "MAJ_n", "PARITY_n", "k_juntas",
    max_metric = 0.0
    counterexample = ""
    instances_tested = 0
    conjecture_holds = True

    for n in n_values:
        for family in families:
            f = generate_function(n, family, seed)
            M, I, W_k = compute_metrics(f, n)
            metric = abs(M) * math.sqrt(1 + I)
            instances_tested += 1
            if metric > max_metric:
                max_metric = metric
            if metric > 2.0:
                conjecture_holds = False
                counterexample = f"n={n}, family={family}, M={M}, I={I}, metric={metric}"

    return {
        "metric_name": "max(|M(f)|*sqrt(1+I(f)))",
        "metric_value": max_metric,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17,
    metric_values = []
    support_fraction = 0.0
    first_failing_seed = None
    counterexample = ""

    for seed in seeds:
```

```
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        metric_values.append(result["metric_value"])
        if result["conjecture_holds"]:
            support_fraction += 1
        else:
            if first_failing_seed is None:
                first_failing_seed = seed
                counterexample = result["counterexample"]

    mean_metric = sum(metric_values) / len(metric_values)
    std_metric = math.sqrt(sum((x - mean_metric) ** 2 for x in metric_values) / len(metric_values)
    support_fraction /= len(seeds)

    if all(result["conjecture_holds"] for result in [run_trial(seed) for seed in seeds]):
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fr
    elif any(not result["conjecture_holds"] for result in [run_trial(seed) for seed in seeds]):
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ac9909ef.py", line 139, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ac9909ef.py", line 114, in run_trial
    M, I, W_k = compute_metrics(f, n)
                ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ac9909ef.py", line 89, in compute_metri
    fourier = fast_walsh_hadamard(f)
              ^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ac9909ef.py", line 38, in fast_walsh_ha
    even = fast_walsh_hadamard(f[:half] + f[half:])
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ac9909ef.py", line 38, in fast_walsh_ha
    even = fast_walsh_hadamard(f[:half] + f[half:])
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ac9909ef.py", line 38, in fast_walsh_ha
    even = fast_walsh_hadamard(f[:half] + f[half:])
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 994 more times]
RecursionError: maximum recursion depth exceeded
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed with a RecursionError in the Walsh-Hadamard transform implementation before producing any data, so neither the support nor falsification criterion can be evaluated. | next: Replace the recursive fast_walsh_hadamard with an iterative in-place implementation (or raise sys.setrecursionlimit and use numpy), then re-run; also explicitly include PARITY_n as a sanity check since |M(PARITY_5)|*sqrt(1+5) = sqrt(6) ≈ 2.449 would immediately falsify.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	318917
2	preregistration	claude_max	opus	0	0	6476
3	novelty	claude_max	opus	0	0	8353
4	novelty	claude_max	opus	0	0	9580
5	test_gen	mistral	codestral-latest	0	0	312714
6	test_gen	mistral	codestral-latest	0	0	311049
7	test_gen	mistral	codestral-latest	0	0	311178
8	test_gen	mistral	codestral-latest	0	0	310728
9	judge	claude_max	opus	0	0	4481

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 1593476 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 4}

(full prompt+response transcripts available in `research/audit/9ea129f53eed.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9ea129f53eed.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/9ea129f53eed.tar.gz` (if generated)